CYBR 437 Secure Coding

Fall 2026

Lab - 5

Release date: October 22, 2026

Due date: October 28, 2026, at 11:59 pm

Introduction

This lab will apply your experience - working with binaries and identifying vulnerable code - with more hands-on exercises. You will convert assembly code back to working C code, find possible memory management issues, and identify buffer and integer overflow vulnerabilities, then revise them appropriately. Specifics for each section can be found below.

Part 1 (30 points):

In this section, you will translate assembly code back to C. Analyze the following assembly code and write some equivalent C source code. You are free to choose your own variable names, and minor variations between your solution and the original are permissible as long as your conversion results in the same disassembled output (for this reason, you may want to compile and review your binary to compare).

For each task, include your C code and detailed explanations on which assembly instructions correspond to which C statements in your code.

*Note: Submissions including **only** the code without appropriate explanations **will not** be eligible for full credit.*

Task 1 (10 points):

```
(gdb) disass
Dump of assembler code for function main:
   0x0000555555555129 <+0>:     endbr64
   0x000055555555512d <+4>:     push   %rbp
   0x000055555555512e <+5>:     mov    %rsp,%rbp
=> 0x0000555555555131 <+8>:     movl   $0x19,-0x4(%rbp)
   0x0000555555555138 <+15>:    movl   $0x1e,-0x8(%rbp)
   0x000055555555513f <+22>:    movq   $0x2d,-0x10(%rbp)
   0x0000555555555147 <+30>:    mov    -0x4(%rbp),%edx
   0x000055555555514a <+33>:    mov    -0x8(%rbp),%eax
   0x000055555555514d <+36>:    add    %edx,%eax
   0x000055555555514f <+38>:    cltq
   0x0000555555555151 <+40>:    sub    -0x10(%rbp),%rax
   0x0000555555555155 <+44>:    mov    %rax,-0x10(%rbp)
   0x0000555555555159 <+48>:    mov    $0x0,%eax
   0x000055555555515e <+53>:    pop    %rbp
   0x000055555555515f <+54>:    ret
End of assembler dump.
```

Task 2 (10 points):

```
(gdb) disass
Dump of assembler code for function main:
   0x0000555555555129 <+0>:     endbr64
   0x000055555555512d <+4>:     push   %rbp
   0x000055555555512e <+5>:     mov    %rsp,%rbp
=> 0x0000555555555131 <+8>:     movl   $0x19,-0x4(%rbp)
   0x0000555555555138 <+15>:    movl   $0x1e,-0x8(%rbp)
   0x000055555555513f <+22>:    mov    -0x4(%rbp),%eax
   0x0000555555555142 <+25>:    cmp    -0x8(%rbp),%eax
   0x0000555555555145 <+28>:    jle    0x555555555150 <main+39>
   0x0000555555555147 <+30>:    movl   $0x1,-0xc(%rbp)
   0x000055555555514e <+37>:    jmp    0x555555555157 <main+46>
   0x0000555555555150 <+39>:    movl   $0x0,-0xc(%rbp)
   0x0000555555555157 <+46>:    mov    $0x0,%eax
   0x000055555555515c <+51>:    pop    %rbp
   0x000055555555515d <+52>:    ret
End of assembler dump.
```

Task 3 (10 points):

```
(gdb) disass
Dump of assembler code for function main:
   0x0000555555555144 <+0>:     endbr64
   0x0000555555555148 <+4>:     push   %rbp
   0x0000555555555149 <+5>:     mov    %rsp,%rbp
=> 0x000055555555514c <+8>:     sub    $0x10,%rsp
   0x0000555555555150 <+12>:    movl   $0x5,-0x4(%rbp)
   0x0000555555555157 <+19>:    movl   $0x6,-0x8(%rbp)
   0x000055555555515e <+26>:    mov    -0x8(%rbp),%edx
   0x0000555555555161 <+29>:    mov    -0x4(%rbp),%eax
   0x0000555555555164 <+32>:    mov    %edx,%esi
   0x0000555555555166 <+34>:    mov    %eax,%edi
   0x0000555555555168 <+36>:    call   0x555555555129 <fun>
   0x000055555555516d <+41>:    mov    %eax,-0x4(%rbp)
   0x0000555555555170 <+44>:    mov    $0x0,%eax
   0x0000555555555175 <+49>:    leave
   0x0000555555555176 <+50>:    ret
End of assembler dump.
(gdb) disass fun
Dump of assembler code for function fun:
   0x0000555555555129 <+0>:     endbr64
   0x000055555555512d <+4>:     push   %rbp
   0x000055555555512e <+5>:     mov    %rsp,%rbp
   0x0000555555555131 <+8>:     mov    %edi,-0x4(%rbp)
   0x0000555555555134 <+11>:    mov    %esi,-0x8(%rbp)
   0x0000555555555137 <+14>:    mov    -0x4(%rbp),%edx
   0x000055555555513a <+17>:    mov    -0x8(%rbp),%eax
   0x000055555555513d <+20>:    add    %edx,%eax
   0x000055555555513f <+22>:    sub    $0x1,%eax
   0x0000555555555142 <+25>:    pop    %rbp
   0x0000555555555143 <+26>:    ret
End of assembler dump.
```

Part 2 (10 points)

Identify and detail any memory management errors in the following code and explain how you will rectify them. Complete the tasks without running the code; no additional code is required on your part.

Task 1 (5 points):

```
#include <stdio.h>
#include <stdlib.h>

int fun(void) {
     char *text_buffer = (char *)malloc(32);
     if (text_buffer == NULL) {
          return -1;
     }
     return 0;
}
```

Task 2 (5 points):

```
#include <stdio.h>
#include <stdlib.h>

int main() {
     int *ptr;
     ptr = (int*)malloc(sizeof(int));
     *ptr = 10;
     printf("%d\n", *ptr);
     free(ptr);
     printf("%d\n", *ptr);
     return 0;
}
```

Part 3 (20 points):

Each of the following C programs/code fragments has potential buffer overflow vulnerabilities. Identify the vulnerabilities, verify them with appropriate test cases (*do not turn off the stack canary during compilation)*, and include the resulting output screen captures in your submission.

Revise the vulnerable code (*considering user inputs/source data invalid is not a valid revision*) to remove the vulnerabilities and explain how your revision eliminates the vulnerabilities. Once secure, re-run the programs with the same test cases to verify the elimination of the vulnerabilities. Include screen captures of the output.

Task 1 (10 points):

```
#include <stdio.h>

int main() {

   char city[10];
   printf("Enter city: ");
   scanf("%s", city);
   printf("City: %s\n", city);

   return 0;
}
```

Task 2 (10 points):

The following function copies `n` elements of the array `from` to the array `to`, starting at position `pos` in array `to`. Assume that the array `from` contains more than `n` elements. *You will need to write an appropriate* `main` *function to call* `fun` *and test.*

```
int fun (double *to, double *from, int pos, int n)
{
   int k;
   for (k=0; k<n; k++)
   {
      to[pos] = from[k];
      pos++;
   }

   return pos;
}
```

Part 4 (40 points):

You will identify potential issues in the given C code and revise the code to address these concerns. Use the header file `limits.h` in C that defines macros to specify the minimum and maximum values of fundamental data types for the specific system and compiler implementation. This allows programmers to use these limits in a portable way without hard-coding system-specific values (you **will not** be hard-coding). To include the said header file, use the preprocessor directive `#include <limits.h>`.

For each task, include your solution with a screen capture of its output, and an explanation of the output.

Task 1 (20 points):

The following function has an integer overflow vulnerability during addition.

```
void fun1 (int a, int b) {
     int sum;
     sum = a + b;
     printf("Result = %d\n", sum);
}
```

- Demonstrate the vulnerability by writing a program and calling this function with two argument values that cause an overflow. The values of the arguments themselves should be within their permissible range. Explain in detail why you chose these values.
- Then rewrite the function to either print a correct result or print an error message if there is a possibility of an overflow.
- Finally, show the execution of the program with the previous values of the arguments that caused an overflow, and with two other values that do not cause an overflow. Explain the output.

Task 2 (10 points):

The following function has an integer overflow vulnerability during subtraction.

```
void fun2 (int x, int y) {
     int diff;
     diff = x - y;
     printf("Result = %d\n", diff);
}
```

- Demonstrate the vulnerability by writing a program and calling this function with two argument values that cause an overflow. The values of the arguments themselves should be within their permissible range. Explain in detail why you chose these values.
- Then rewrite the function to either print a correct result or print an error message if there is a possibility of an overflow.
- Finally, show the execution of the program with the previous values of the arguments that caused an overflow, and with two other values that do not cause an overflow. Explain the output.

Submission Instructions

Your write-up submission, which should include:

- Your converted source code with detailed comments/explanation for Part 1.
- Explanations for the tasks in Part 2.
- The code, explanations, and screenshots of the outputs for the tasks in Parts 3 and 4.

Include your written source file(s) for Parts 1, 3, and 4 (all of them can be in one file as long as they are separated/commented appropriately for each task).